

StatTraveledDistance [SimTalk] - Worker

Returns the distance which the instance of the *Worker* designated by <Path> covered while walking from the *WorkerPool* to the *Workplaces* attached to the stations and between the stations in meters.

Type

Read-only attribute

Syntax

```
<Path>.StatTraveledDistance → length
```

Return Value

The return value has the data type *length*.

Example

```
print .Resources.MyWorker:2.StatTraveledDistance
```

See also

[Tab Statistics \[Worker\]](#)

Statistics report, [Worker Statistics — Traveled Distance by Workers](#)

Attributes of the Worker

Attributes of the Worker

The *Worker* provides:

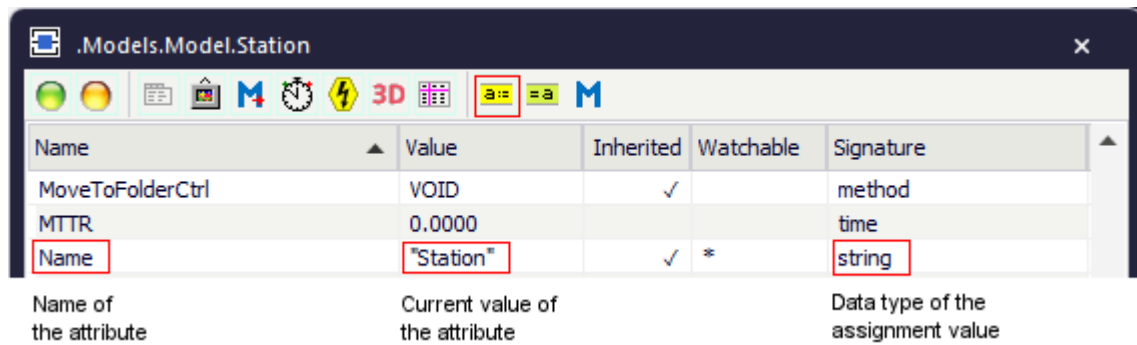
- The *attributes* listed in the table of contents to the left.

Note

If you do not use a [FootPath](#), *Plant Simulation* beams the *Worker* to the *Workplace*.

- The [_Attributes of the Exporter](#) as the *Worker* is an [Exporter \[object\]](#) with the capacity of 1.
- The [Attributes of All Objects](#).

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window [Show Attributes and Methods](#). The figure below illustrates the information using the example of the object *Station*.



- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected [Class \[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected [Instance \[general description\]](#).

You can set the value of an attribute and you can get its value, either with the check boxes, the text boxes and drop-down lists in the dialog windows or by assigning values to the respective attributes.

- To **set the value** of an attribute, you might, for example, type:

```
.Resources.MyWorker:2.Efficiency := 90
```

- To **get the value** of an attribute, you might, for example, type:

```
print .Resources.Worker.MyWorker:2.Efficiency
posit := Station.Cont.XPos
```

AutomaticMediation [SimTalk] - Worker

Prevents that the *Worker* designated by <Path>, which you yourself want to assign, is automatically assigned by the *Broker*.

Remarks

If no *Broker* exists when creating the *Workers*, *Plant Simulation* sets the attribute *AutomaticMediation* of the *Workers* to **false**.

A *Worker*, who is not automatically brokered, does not return to the *WorkerPool* when his state changes to paused or unplanned.

Note

If *AutomaticMediation* is deactivated, and if the method *goTo* is called when the parameter *ReserveExclusively* has the value `true`, meaning that it reserves the *Workplace*, *Plant Simulation* checks if the *Workplace* still has available **Capacity**. If all places of the *Workplace* are either occupied or reserved, *Plant Simulation* shows an error message.

Type

Attribute

Syntax

```
<Path>.AutomaticMediation:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Specify `true` to make the *Broker* assign the *Worker* automatically.

Specify `false` to assign the *Worker* yourself.

Example

```
.Resources.MyWorker:2.AutomaticMediation := true
param type: integer // Importer type (0=failure, 1=setup, 2=processing,
3=transport)
var t: table
switch type
case 0
    ?.failImp.releaseExporters
case 1
```

```
?setupImp.getExporters(t)
if t.yDim > 0
    t[1,1].AutomaticMediation := true
end
?setupImp.releaseExporters
case 2
    ?imp.getExporters(t)
    if t.yDim > 0
        t[1,1].AutomaticMediation := true
    end
    ?imp.releaseExporters
end
```

SimTalk

goTo [SimTalk]

importExporter [SimTalk]

BrokerPath [SimTalk] - Worker

Sets the path to the *Broker* who brokers the services that the *Worker* designated by <Path> provides.

Type

Attribute

Syntax

```
<Path>.BrokerPath:object
```

Assignment Value

You can assign a value of data type *object*.

Example

```
print .Resources.MyWorker:2.BrokerPath  
.Resources.MyWorker:2.BrokerPath := myBroker  
.Resources.MyWorker:2.BrokerPath := Broker3
```

See also

[Broker \[Worker\]](#)

Efficiency [SimTalk] - Worker

Sets the **Efficiency** of the *Worker* designated by <Path>, i.e., how fast he performs the job assigned to him.

Type

Attribute

Syntax

```
<Path>.Efficiency:integer
```

Assignment Value

You can assign a value of data type *integer*.

- With an **Efficiency** of 100 percent the *Worker* consumes the exact processing time which you specified.
- With an **Efficiency** of 200 percent he consumes half of the processing time which you specified.
- With an **Efficiency** of 50 percent he consumes twice the processing time which you specified.

Note

If more than one *Worker* performs jobs at the station, the efficiency of the slowest *Worker* determines the processing time of the *MU*.

Example

```
.Resources.MyWorker:2.Efficiency := 90
```

See also

[Efficiency \[text box\]](#)

[Efficiency \[SimTalk\] - WorkerPool](#) in the [Workers to Create](#)

HomeLocation [SimTalk] - Worker

Sets the path to the **Home Location**, i.e., the *Workplace* to which the *Worker* designated by <Path> walks, once he has finished his current job.

Remarks

Plant Simulation transfers *Workers*, for whom you defined a **Home Location** and a **Shift** in the **Workers to Create table** of the *WorkerPool*, to the defined **Home Location** at the start of the **Shift**. They then start walking to the respective *Workplace* from there.

At the end of the shift *Plant Simulation* transfers all *Workers* of this shift directly to the *WorkerPool*, they do not walk back to the *WorkerPool*.

Plant Simulation transfers *Workers*, for whom you defined a **Home Location** but no **Shift**, to the defined **Home Location** at the start of the simulation.

Note

Plant Simulation always resolves **relative paths** in relation to the *Frame* in which the *WorkerPool* is inserted. If *Plant Simulation* cannot resolve a path in the **Scope** array during the simulation, it shows an error message and asks if you would like to stop the simulation.

Type

Attribute

Syntax

```
<Path>.HomeLocation:object
```

Assignment Value

You can assign a value of data type *object*.

Example

```
.Resources.MyWorker:2.HomeLocation := ~.Models.Model.Workplace  
print .Resources.MyWorker:2.HomeLocation
```

SimTalk

setWorkersToCreateTable [SimTalk]

See also

[Home Location](#)

[Workers to Create](#)

[The Relative Path](#)

IsIdle [SimTalk] - Worker

Sets if the *Worker* designated by <Path> is idle (`true`), i.e., does not work, or is not idle (`false`).

Remarks

IsIdle works together with the read-only attribute *NumIdleWorkers* of the *WorkerPool*.

NumIdleWorkers returns the number of *Workers* for which the attribute *IsIdle* returned `true`.

The method *getIdleWorker* of the *WorkerPool* returns the first idle *Worker* for whom *IsIdle* is `true` and sets it to `false`. When you no longer need the *Worker*, you can send the *Worker* to the *WorkerPool* and set *IsIdle* to `true`.

Type

Attribute

Syntax

```
<Path>.IsIdle:boolean
```

Watchable

The attribute is [watchable](#).

Assignment Value

You can assign a value of data type *boolean*.

Example

```
.Resources.Worker:1.IsIdle := true
```

SimTalk

[NumIdleWorkers \[SimTalk\]](#)

[getIdleWorker \[SimTalk\]](#)

OrderCtrl [SimTalk] - Worker

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* whenever the *Worker* is assigned to an *importer*.

In the control you determine how the *Worker* handles an order.

Type

Attribute

Syntax

```
<Path>.OrderCtrl:method
```

Parameters

The **Order Control** has two parameters that define the *importer*:

- The parameter *Importer* of data type *object* designates the *importer*.
- The parameter *Type* of data type *integer* designates its **type**: 0 designates the *failure/remove failure-importer*, 1 the *set-up-importer*, 2 the *processing-importer*, and 3 the *transport-importer*.

Reformatting the Method

If you manually enter an **Order Control** and click **Apply** or **OK**, *Plant Simulation* automatically checks if the *Method* expects the correct parameters. If this is not the case, *Plant Simulation* shows a message asking if the *Method* shall be reformatted or not. Empty methods are reformatted automatically. You cannot suppress checking of the format of the *Method*.

Assignment Value

You can assign a value of data type *method*.

Example

```
.Resources.MyWorker:2.OrderCtrl := &myOrderCtrl
```

See also

[Order Control \[Worker\]](#)

Priority [SimTalk] - Worker

Sets the priority with which the *Worker* designated by <Path> carries out a work order.

Remarks

The *Priority* of a *Worker* is a criterion for the urgency of a request. The *Priority* is an integer value.

Workers with a higher **Priority** will be brokered earlier by the *Broker* for identical services than *Workers* with a lower priority. The **Priority** only applies to *Workers* who are registered with the same *Broker*.

If several qualified *Workers* with the same priority exist, who can provide the requested service, *Plant Simulation* first brokers the *Workers*, who already stay on a suitable *Workplace* at the station. After that *Workers* are brokered who already are at the station, but stay on the wrong *Workplace*. Then *Workers* are brokered who are not staying on a *Workplace*. Finally, those *Workers* are brokered who stay on a *Workplace* that is assigned to another station.

Type

Attribute

Syntax

```
<Path>.Priority:integer
```

Assignment Value

You can assign a value of data type *integer*.

The higher the value that you enter, the higher is the priority. *Plant Simulation* provides a *Worker* with *Priority* 10 before providing a *Worker* with *Priority* 1.

Example

```
.Resources.MyWorker:2.Priority := 6
```

See also

[Priority \[Worker\]](#)

ReleaseCtrl [SimTalk] - Worker

Designates a *Method* of the *Worker* designated by <Path>.

Remarks

The *Worker* executes the *control* as soon as an *importer* releases the *Worker*. When the control is called, the *Worker* has already left its *importer*. If you did enter a *Method*, the *Worker* will not be brokered for already waiting *importers*. To broker the *Worker* to already waiting *importers* nonetheless, you have to use the method [findNewImporter](#).

Syntax

```
<Path>.ReleaseCtrl:method
```

Parameters

The **Release Control** has two parameters that define the *importer*:

- The parameter *Importer* of data type *object* designates the *importer* proper.
- The parameter *Type* of data type *integer* designates its type: 0 designates the *failure/remove failure-importer*, 1 the *set-up-importer*, 2 the *processing-importer*, and 3 the *transport-importer*.

Note

Plant Simulation calls the **Release Control**, which you typed into the *Worker* for the *transport-importer*, when the *Worker* starts to carry parts away. This is then the case if either the **Maximum Dwell Time** has passed or if the *Worker* cannot pick up any more parts because his **X-Dimension**, **Y-Dimension**, and **Z-Dimension** is used up.

Reformatting the Method

If you manually enter a **Release Control** and click **Apply** or **OK**, *Plant Simulation* automatically checks if the *Method* expects the correct parameters. If this is not the case, *Plant Simulation* shows a message asking if the *Method* shall be reformatted or not. Empty methods are reformatted automatically. You cannot suppress checking of the format of the *Method*.

Assignment Value

You can assign a value of data type *method*.

Example

```
.Resources.MyWorker:2.ReleaseCtrl := &myWorkerReleaseCtrl
```

SimTalk

[findNewImporter \[SimTalk\]](#)

See also

[Release Control \[Worker\]](#)

Scope [SimTalk] - Worker

Sets the **Scope** of the *Worker* designated by <Path>. The **Scope** array contains all objects to which the *Worker* can be brokered to do a job.

Remarks

Plant Simulation always resolves relative paths in relation to the *Frame* in which the *WorkerPool* is inserted. If *Plant Simulation* cannot resolve a path in the **Scope** array during the simulation, it shows an error message and asks if you would like to stop the simulation.

To allow the *Worker* to be brokered to a *Frame*, including all of the inserted objects and *sub-Frames*, specify that *Frame*.

Type

Attribute

Syntax

```
<Path>.Scope:array[]
```

Assignment Value

You can assign an *array* of data type *string*.

Examples

```
// gets the scope
var Scope: object[] := MyWorker.Scope // array of objects
```

```
// deletes the list of objects of the scope
MyWorker.Scope := VOID
```

```
// deletes the list of objects of the scope
MyWorker.Scope := []
```

```
// replaces the list of objects of the scope with an object
MyWorker.Scope := .Models.Model.Station
```

```
// replaces the list of objects of the scope with a list of objects
MyWorker.Scope := [.Models.Model.Station, .Models.Model.ParallelStation]
```

See also

Tab Scope

Services [SimTalk] - Worker

Sets the names of the **Services** that the *Worker* designated by <Path> provides.

Remarks

The name is not case-sensitive, just like the names of *attributes* and *methods* of the objects are not case-sensitive.

To save memory and improve access speed, all places which are using such a case-insensitive string are pointing to the same string in main memory. The visible and unexpected result is that the first occurrence of the string defines how the string is written in terms of upper- and lower-casing.

In *SimTalk* you can compare strings in an case-insensitive manner with the `~=` operator, compare [Relational Operators](#).

Syntax

```
<Path>.Services:array[]
```

Assignment Value

You can assign an *array* of data type *string*.

Examples

```
var a : string[] := ["Job1", "Job2", "Job3"]  
.Resources.MyWorker:1.Services := a
```

```
print .Resources.MyWorker:1.Services
```

See also

[Services \[Worker\]](#)

[Relational Operators](#)

Shift [SimTalk] - Worker

Sets the name of the **Shift** during which the *Worker* designated by <Path> works.

Remarks

Specify an empty string "" to make the *Worker* work during all shifts which are defined in the associated *ShiftCalendar*.

Syntax

```
<Path>.Shift:string
```

Assignment Value

You can assign a value of data type *string*.

Example

```
.Resources.MyWorker:2.Shift := "Night shift"  
.Resources.MyWorker:3.Shift := "" // the worker works during all shifts
```

See also

[Shift \[Worker\]](#)

Speed [SimTalk] - Worker

Sets the **Speed** with which the *Worker* designated by <Path> walks on the *FootPath* or walks freely within the area to his *Workplace*.

Type

Attribute

Syntax

```
<Path>.Speed: speed
```

Assignment Value

You can assign a value of data type *speed*.

Note

In *SimTalk* 2.0 you can specify the speed units mps, fps, kmh, and mph.

Type in the unit directly after the number, without a separating blank space, for example 100kmh or 100.5kmh.

```
var len := 1.0ft  
var s : speed := 10.5m / 1:30  
var x : length := 3m
```

Example

```
.Resources.MyWorker:2.Speed := 0.5
```

See also

[Speed \[Worker\]](#)

Stopped [SimTalk] - Worker

Stops the *Worker* designated by <Path> on his way to or from the destination (`true`) or makes him continue on his way (`false`).

Remarks

If the *Worker* stays in the *WorkerPool* or on a *Workplace*, you cannot stop him.

Type

Attribute

Syntax

```
<MU-Path>.Stopped: boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
.UserObjects.MyWorker:2.Stopped := true    // Worker stops  
.UserObjects.MyWorker:2.Stopped := false   // Worker continues
```

See also

[Stopped \[check box\] - Worker](#)

WorkerPool [SimTalk]

Sets the *WorkerPool* to which the *Worker* designated by <Path> belongs.

Syntax

```
<Path>.WorkerPool:object
```

Assignment Value

You can assign a value of data type *object*.

Example

```
.Resources.MyWorker:2.WorkerPool := MyWorkerPool
```

XDim [SimTalk] - Worker

Sets the number of *parts* that the *Worker* designated by <Path> can carry in the **X-Dimension**.

Remarks

The carrying capacity of the *Worker* is the product of *XDim* times *YDim* times *XDim*. The greatest allowed value is one million.

If you decrease the carrying capacity of the *Worker*, make sure that no *parts* are located on the places that will be deleted by this action! Either delete these *parts* or move them to another place on the reduced carrying capacity area.

Type

Attribute

Syntax

```
<Path>.XDim:integer
```

Watchable

The attribute is **watchable**.

Assignment Value

You can assign a value of data type *integer*.

Example

```
.Resources.Worker.XDim := 2
```

SimTalk

[YDim \[SimTalk\] - Worker](#)

[ZDim \[SimTalk\] - Worker](#)

See also

[X-Dimension \[Worker\]](#)

YDim [SimTalk] - Worker

Sets the number of *parts* that the *Worker* designated by <Path> can carry in the **Y-Dimension**.

Remarks

The carrying capacity of the *Worker* is the product of *XDim* times *YDim* times *XDim*. The greatest allowed value is one million.

If you decrease the carrying capacity of the *Worker*, make sure that no *parts* are located on the places that will be deleted by this action! Either delete these *parts* or move them to another place on the reduced carrying capacity area.

Type

Attribute

Syntax

```
<Path>.YDim:integer
```

Watchable

The attribute is **watchable**.

Assignment Value

You can assign a value of data type *integer*.

Example

```
.Resources.Worker.YDim := 2
```

SimTalk

[XDim \[SimTalk\] - Worker](#)

[ZDim \[SimTalk\] - Worker](#)

See also

[X-Dimension \[Worker\]](#)

ZDim [SimTalk] - Worker

Sets the number of *parts* that the *Worker* designated by <Path> can carry in the **Z-Dimension**.

Remarks

The carrying capacity of the *Worker* is the product of *XDim* times *YDim* times *XDim*. The greatest allowed value is one million.

The **Z-Dimension** enables stacking *parts* one onto the other. The stacked *parts* are only visible in 3D.

If you decrease the carrying capacity of the *Worker*, make sure that no *parts* are located on the places that will be deleted by this action! Either delete these *parts* or move them to another place on the reduced carrying capacity area.

Type

Attribute

Syntax

```
<Path>.ZDim:integer
```

Watchable

The attribute is [watchable](#).

Assignment Value

You can assign a value of data type *integer*.

Example

```
.Resources.Worker.YDim := 2
```

SimTalk

[XDim \[SimTalk\] - Worker](#)

[YDim \[SimTalk\] - Worker](#)

See also

[X-Dimension \[Worker\]](#)

Exporter

Exporter [object]

Use the object *Exporter* for providing and exporting services. It represents a group of people, whose individual members you cannot distinguish and whom you cannot address as individuals.

Image



Description

The *Exporter* works together with the **Broker** and the *Importers* (see **Tab Importer** and **Sub-tab Failure**) of the *Station*, the *ParallelStation*, the *AssemblyStation*, and the *DismantleStation*. The *Exporter* offers services and provides them for *Importers*.

A single *Broker* manages the *Exporter* and assigns it to an *Importer*. After the *Exporter* has finished providing its service, it registers as being available with its *Broker* which will assign it to other *Importers* as soon as its services are required.

Note

If the *Importer* requests several services at the same time, all of these services have to be available at the same time, before the *Broker* assigns them to the station.

Think of the *Exporter* as a group of people, whose individual members you cannot distinguish and whom you cannot address as individuals. To distinguish individual staff members, you have to model them with individual *Exporters* or **Workers**. An *Exporter* with a **capacity** greater than 1 can simultaneously provide services at several *Importers*. You can use the *Exporter* if transit times for traveled distances are not important for the simulation.

Note

An *Exporter* may provide services for several *Importers* at the same time.

To show a **tooltip** with information about the *Exporter*, hover with the mouse over it.